

UNO Vision — IAPR 2026 Final Report

Task: For each UNO game image (4000×2662 px), predict:

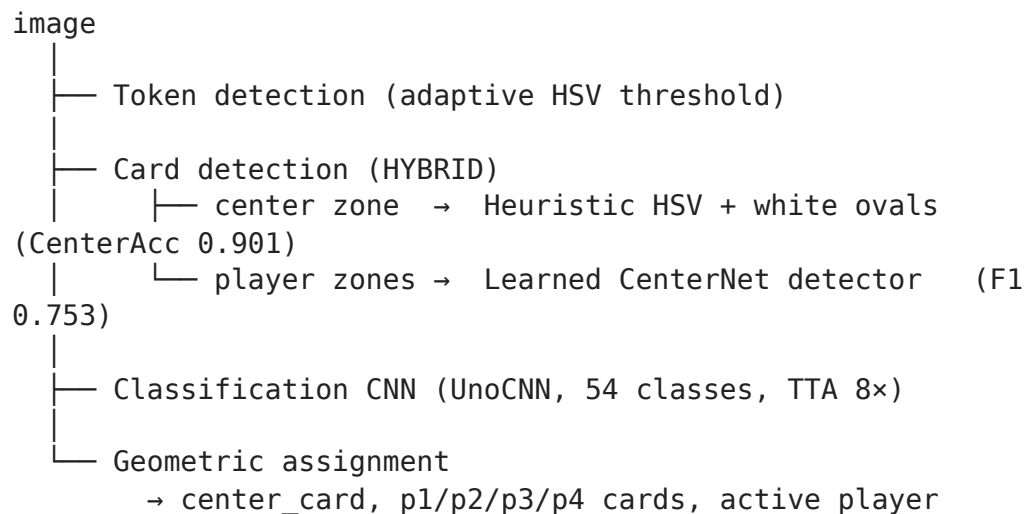
- The **center card** played in the middle
- The **active player** (p1/p2/p3/p4) — the one holding the token
- The **hand cards** of each of the 4 players

Metric: $\text{Score} = 0.1 \cdot \text{CenterAcc} + 0.1 \cdot \text{ActiveAcc} + 0.8 \cdot \text{F1}$ (multi-set F1 over player cards)

Constraints:

- No external datasets or pre-trained models
- $\leq 12\text{M}$ parameters total across all models
- 81 annotated training images + 159 test images

Pipeline overview



```

In [2]: import sys
        from pathlib import Path

        import cv2
        import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
        import matplotlib.patches as mpatches
        import torch

        ROOT = Path('..').resolve()
        sys.path.insert(0, str(ROOT))
  
```

```

from src.config import (
    CARD_CLASSES, NUM_CLASSES, TRAIN_DIR, TRAIN_CSV,
    REFERENCE_DIR, DATA_DIR, MODELS_DIR
)

TEMPLATES_DIR = DATA_DIR / 'card_templates'
CLASSIFIER_PT = MODELS_DIR / 'classifier.pt'
DETECTOR_PT = MODELS_DIR / 'detector.pt'

print(f'Train images : {len(list(TRAIN_DIR.glob("*.jpg")))}')
print(f'Card classes : {NUM_CLASSES}')
print(f'Classifier : {CLASSIFIER_PT.exists()}')
print(f'Detector : {DETECTOR_PT.exists()}')

```

```

Train images : 81
Card classes : 54
Classifier : True
Detector : True

```

1. Dataset

81 annotated training images. Each row of `train.csv` gives the center card, active player, and hand cards for all 4 players.

```

In [3]: df = pd.read_csv(TRAIN_CSV)
print(f'Shape: {df.shape}')
df.head(5)

```

Shape: (81, 7)

```

Out[3]:

```

	image_id	center_card	active_player	player_1_cards	player_2_cards	player_3_cards
0	L1000770	y_1	p3	EMPTY	y_4;g_2;r_3	r_skip;y_
1	L1000771	r_5	p1	y_7;y_8;g_4	EMPTY	r_
2	L1000772	r_9	p2	y_8;b_6;g_3	b_skip;r_0;r_3	r_6;g_
3	L1000773	y_8	p4	r_1;wild;g_2	EMPTY	draw_4;g_revers
4	L1000774	r_4	p3	g_4;g_8;r_7;b_3	r_9;g_3	wild;y_

```

In [4]: # Class distribution across all player hands
all_cards = []
for col in ['player_1_cards', 'player_2_cards', 'player_3_cards', 'player_4_cards']:
    for hand in df[col].dropna():
        if hand != 'EMPTY':
            all_cards.extend(hand.split(';'))

from collections import Counter
counts = Counter(all_cards)
labels_sorted = sorted(counts.keys())
values = [counts[l] for l in labels_sorted]

```

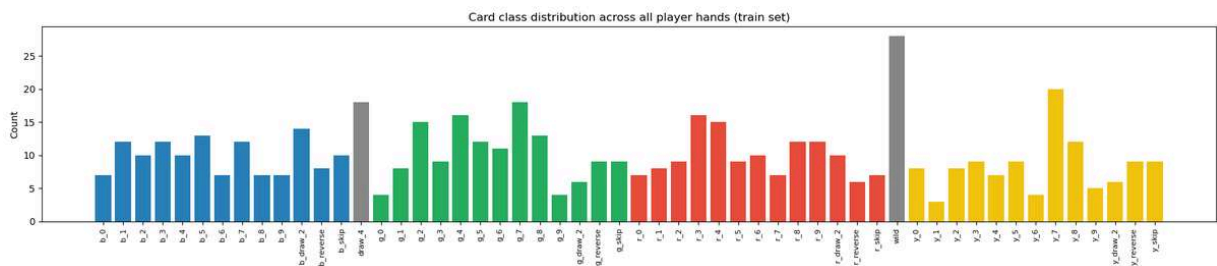
```

color_map = {'r': '#e74c3c', 'g': '#27ae60', 'b': '#2980b9', 'y': '#f1c40f',
             'w': '#888888', 'd': '#888888'}
bar_colors = [color_map.get(l[0], '#888888') for l in labels_sorted]

fig, ax = plt.subplots(figsize=(18, 4))
ax.bar(range(len(labels_sorted)), values, color=bar_colors)
ax.set_xticks(range(len(labels_sorted)))
ax.set_xticklabels(labels_sorted, rotation=90, fontsize=8)
ax.set_title('Card class distribution across all player hands (train set)')
ax.set_ylabel('Count')
plt.tight_layout()
plt.show()

print(f'Total card instances in training annotations: {sum(values)}')

```



Total card instances in training annotations: 546

```

In [5]: # Show 4 training scenes
sample_ids = df['image_id'].tolist()[:4]
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
for ax, img_id in zip(axes, sample_ids):
    img = cv2.imread(str(TRAIN_DIR / f'{img_id}.jpg'))
    ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    row = df[df.image_id == img_id].iloc[0]
    ax.set_title(f'{img_id}\ncenter={row.center_card} active={row.active_pl')
    ax.axis('off')
plt.suptitle('Sample training images', fontsize=12)
plt.tight_layout()
plt.show()

```



2. Template extraction

The 54 UNO card templates are extracted from the official `reference_images` via HSV segmentation + color-based matching. These form the seed of our synthetic training dataset.

```

In [7]: template_files = sorted(TEMPLATES_DIR.glob('*.png')) if TEMPLATES_DIR.exists:
print(f'Templates found: {len(template_files)}')

if template_files:
    # Build a grid: 9 cols x 6 rows
    cols, rows = 9, 6
    thumb_w, thumb_h = 100, 150
    canvas = np.ones((rows * thumb_h + rows * 4, cols * thumb_w + cols * 4,
                      dtype=np.uint8) * 240
    for i, f in enumerate(template_files[:cols * rows]):
        r, c = divmod(i, cols)
        img = cv2.imread(str(f))
        img = cv2.resize(img, (thumb_w, thumb_h))
        y0, x0 = r * (thumb_h + 4), c * (thumb_w + 4)
        canvas[y0:y0+thumb_h, x0:x0+thumb_w] = img

    fig, ax = plt.subplots(figsize=(18, 12))
    ax.imshow(cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB))
    ax.axis('off')
    ax.set_title('54 UNO card templates extracted from reference_images', fontweight='bold')
    plt.tight_layout()
    plt.show()
else:
    print('Run scripts/extract_templates.py first to generate card_templates')

```

Templates found: 54

54 UNO card templates extracted from reference_images



3. Architecture

3.1 Classifier — UnoCNN (ResNet-18-light, 11.20M params)

Input: 144×96 RGB crop

Stem : Conv3×3 3→64 (no stride – preserve local features)

Stage1 : 2× BasicBlock 64→64

Stage2 : 2× BasicBlock 64→128 (stride 2)

Stage3 : 2× BasicBlock 128→256 (stride 2)

Stage4 : 2× BasicBlock 256→512 (stride 2)

Head : GAP → Dropout(0.3) → Linear(512, 54)

Output: 54 logits (one per UNO card class)

BasicBlock = Conv3×3 + BN + ReLU + Conv3×3 + BN + residual skip. Skip connections prevent gradient vanishing — crucial for distinguishing visually similar classes (e.g.

y_8 vs y_9 , b_skip vs b_reverse).

3.2 Detector — CenterNet-style anchor-free (0.53M params)

Input: 512×352 image

Stem : Conv3×3 3→16 (stride 2) → 256×176

Stage1: 2× BasicBlock 16→32 (stride 2) → 128×88

Stage2: 2× BasicBlock 32→64 (stride 2) → 64×44

Stage3: 2× BasicBlock 64→96 (stride 2) → 32×22

Head : Conv3×3 96→64

└─ head_obj : Conv1×1 64→1 (objectness heatmap, logits)

└─ head_bbox : Conv1×1 64→4 (cx_norm, cy_norm, w_norm, h_norm → sigmoid)

Trained on 626 manually annotated bboxes (LabelImg, YOLO format). Loss = Focal($\alpha=2, \beta=4$) on heatmap + L1 on bbox at peak pixel.

Total: 0.53M + 11.20M = 11.73M < 12M limit ✓

```
In [8]: from src.model import UnoCNN, count_parameters as count_clf
        from src.detector import CardDetector, count_parameters as count_det

        clf_model = UnoCNN()
        det_model = CardDetector()

        n_clf = count_clf(clf_model)
        n_det = count_det(det_model)
        n_total = n_clf + n_det

        print(f'Classifier (UnoCNN)      : {n_clf:>12,} params')
        print(f'Detector (CenterNet)    : {n_det:>12,} params')
        print(f'Total                  : {n_total:>12,} params')
        print(f'Under 12M limit           : {n_total < 12_000_000}')

        # Verify forward pass shapes
        x_clf = torch.randn(2, 3, 144, 96)
        x_det = torch.randn(1, 3, 352, 512)
        y_clf = clf_model(x_clf)
        y_det = det_model(x_det)
        print(f'\nClassifier : {tuple(x_clf.shape)} → {tuple(y_clf.shape)}')
        print(f'Detector obj: {tuple(x_det.shape)} → {tuple(y_det["obj"].shape)}')
        print(f'Detector box: {tuple(x_det.shape)} → {tuple(y_det["bbox"].shape)}')
```

```
Classifier (UnoCNN)      : 11,196,534 params
Detector (CenterNet)    : 532,245 params
Total                  : 11,728,779 params
Under 12M limit         : True

Classifier : (2, 3, 144, 96) → (2, 54)
Detector obj: (1, 3, 352, 512) → (1, 1, 22, 32)
Detector box: (1, 3, 352, 512) → (1, 4, 22, 32)
```


4. Augmentation strategy

The classifier is trained **from scratch** on synthetic crops generated from the 54 templates — no real crop annotations available initially. Augmentations simulate what the detector will pass downstream:

Transform	Range / Prob
Discrete rotation	{0°, 90°, 180°, 270°}
Fine tilt	±15°
Perspective jitter	±12 px per corner (p=0.7)
Scale	0.88–1.10×
Brightness	±50
Contrast	×0.65–1.35
Saturation	×0.55–1.45
Hue	±8°
Gaussian blur	$\sigma \leq 2.0$ (p=0.5)
Gaussian noise	$\sigma \leq 14$ (p=0.6)
BG fringe	60% — card shrunk to 82–98%, placed on white/noisy synthetic bg

BG fringe is critical: detections often capture a few pixels of background at borders. Without it, the classifier overfits to clean centered templates and fails on real crops.

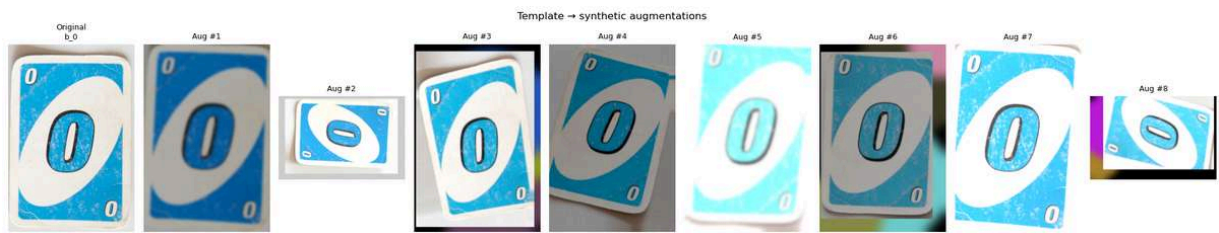
```
In [9]: from src.augmentation import augment, AugConfig

# Pick one template to visualize augmentations
tpl_files = sorted(TEMPLATES_DIR.glob('*.png')) if TEMPLATES_DIR.exists() else []

if tpl_files:
    rng = np.random.default_rng(42)
    template = cv2.imread(str(tpl_files[0])) # e.g. b_0
    cfg = AugConfig()

    n_aug = 8
    fig, axes = plt.subplots(1, n_aug + 1, figsize=(20, 4))
    axes[0].imshow(cv2.cvtColor(template, cv2.COLOR_BGR2RGB))
    axes[0].set_title(f'Original\n{tpl_files[0].stem}', fontsize=9)
    axes[0].axis('off')
    for i in range(n_aug):
        aug = augment(template, rng, cfg)
        axes[i+1].imshow(cv2.cvtColor(aug, cv2.COLOR_BGR2RGB))
        axes[i+1].set_title(f'Aug #{i+1}', fontsize=9)
        axes[i+1].axis('off')
    plt.suptitle('Template → synthetic augmentations', fontsize=12)
```

```
plt.tight_layout()
plt.show()
else:
    print('Run scripts/extract_templates.py first.')
```



5. Auto-labeling of real crops (key contribution)

A classifier trained only on synthetic crops fails on real game images: the distribution gap (bg leakage, blur, reflections) is too large.

Solution: use the 81 GT-annotated train images to auto-label real crops:

1. Detect cards in each train image via heuristic
2. Assign each detected crop to its player via geometry
3. Match the crop to its GT card label via dominant color similarity
4. Save 389 validated real crops in `data/real_crops/<label>/`

These are mixed 50/50 with synthetic crops during training. **F1 went from 0.42 → 0.69** with this addition.

```
In [26]: REAL_CROPS_DIR = DATA_DIR / 'real_crops'

if REAL_CROPS_DIR.exists():
    all_real = list(REAL_CROPS_DIR.glob('*/*.png'))
    label_counts = Counter(f.parent.name for f in all_real)
    labels_r = sorted(label_counts.keys())
    values_r = [label_counts[l] for l in labels_r]
    bar_colors_r = [color_map.get(l[0], '#888888') for l in labels_r]

    fig, ax = plt.subplots(figsize=(18, 3))
    ax.bar(range(len(labels_r)), values_r, color=bar_colors_r)
    ax.set_xticks(range(len(labels_r)))
    ax.set_xticklabels(labels_r, rotation=90, fontsize=8)
    ax.set_title(f'Real crops per class - {len(all_real)} total auto-labeled')
    ax.set_ylabel('Count')
    plt.tight_layout()
    plt.show()

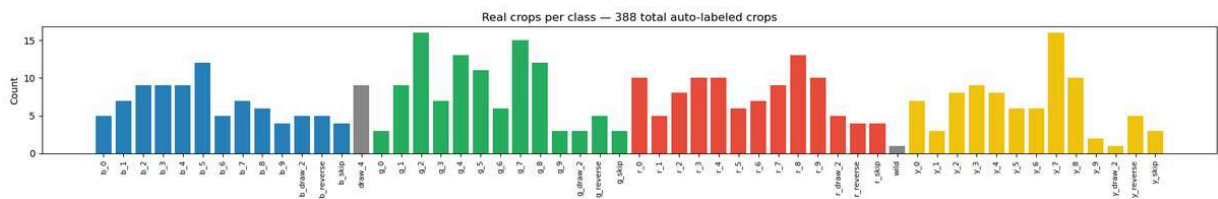
    # Show a few real crops vs their template
    sample_labels = [l for l in labels_r if label_counts[l] >= 3][:5]
    n_show = 4
    fig, axes = plt.subplots(len(sample_labels), n_show + 1, figsize=(14, 3*
    for row_i, lbl in enumerate(sample_labels):
        tpl_path = TEMPLATES_DIR / f'{lbl}.png'
        tpl_img = cv2.imread(str(tpl_path)) if tpl_path.exists() else np.zeros(
```



```

axes[row_i][0].imshow(cv2.cvtColor(tpl_img, cv2.COLOR_BGR2RGB))
axes[row_i][0].set_title(f'{lbl}\n(template)', fontsize=8)
axes[row_i][0].axis('off')
crops = sorted((REAL_CROPS_DIR / lbl).glob('*.png'))[:n_show]
for j, cp in enumerate(crops):
    cimg = cv2.imread(str(cp))
    axes[row_i][j+1].imshow(cv2.cvtColor(cimg, cv2.COLOR_BGR2RGB))
    axes[row_i][j+1].set_title(f'real #{j+1}', fontsize=8)
    axes[row_i][j+1].axis('off')
for j in range(len(crops), n_show):
    axes[row_i][j+1].axis('off')
plt.suptitle('Template vs auto-labeled real crops', fontsize=12)
plt.tight_layout()
plt.show()
else:
    print('Run scripts/autolabel_train.py first to generate data/real_crops/

```





6. Loading the trained models

```
In [11]: from src.inference import Classifier, predict_scene
from src.detector_inference import CardDetectorRuntime

classifier = Classifier(CLASSIFIER_PT)
detector   = CardDetectorRuntime(DETECTOR_PT, device=str(classifier.device))

print(f'Classifier loaded - device: {classifier.device}')
print(f'Detector loaded - device: {classifier.device}')
```

Classifier loaded – device: cpu
 Detector loaded – device: cpu

7. Card detection visualization

The detection is **hybrid**: heuristic HSV on the center zone, learned CenterNet on player zones.

- **Heuristic** (white background): Gaussian blur on HSV saturation → threshold → contours → rotated rects. Stacks split by k-means color.
- **Heuristic** (noisy background): enclosed white ovals in the saturation mask → infer card rect by scaling (×1.55).
- **Learned detector**: anchor-free CenterNet on 512×352 resized input → objectness heatmap + bbox regression → NMS.

```
In [12]: from src.detection import detect_cards_in_scene, detect_active_token
from src.detector_inference import detect_cards_with_model
from src.inference import predict_scene

def draw_detections(image, cards, token=None, title=''):
    vis = image.copy()
    for c in cards:
        box = cv2.boxPoints(c['rect']).astype(np.intp)
        cv2.drawContours(vis, [box], 0, (0, 220, 0), 6)
        cx, cy = int(c['cx']), int(c['cy'])
        cv2.circle(vis, (cx, cy), 10, (0, 220, 0), -1)
    if token:
        tx, ty = int(token[0]), int(token[1])
        cv2.circle(vis, (tx, ty), 60, (255, 140, 0), 10)
        cv2.circle(vis, (tx, ty), 10, (255, 140, 0), -1)
    return vis

# Pick one white-bg and one noisy-bg image
sample_pairs = [('L1000770', 'White background'), ('L1000909', 'Noisy backgr

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
for ax, (img_id, bg_type) in zip(axes, sample_pairs):
    img_path = TRAIN_DIR / f'{img_id}.jpg'
    if not img_path.exists():
        ax.set_title(f'{img_id} not found – download data first')
        ax.axis('off')
        continue
    img = cv2.imread(str(img_path))
    token = detect_active_token(img)
    cards_heur = detect_cards_in_scene(img, exclude_xy=token)
    vis = draw_detections(img, cards_heur, token)
    ax.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))
    ax.set_title(f'{bg_type} – {img_id}\nHeuristic: {len(cards_heur)} cards
    ax.axis('off')

plt.suptitle('Heuristic card detection (green boxes) + token (orange)', font
```

```
plt.tight_layout()
plt.show()
```



```
In [13]: # Learned detector heatmap visualization
from src.detector_dataset import INPUT_H, INPUT_W

img_id = 'L1000770'
img_path = TRAIN_DIR / f'{img_id}.jpg'

if img_path.exists():
    img = cv2.imread(str(img_path))
    img_resized = cv2.resize(img, (INPUT_W, INPUT_H))
    img_rgb = cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB)
    tensor = torch.from_numpy(img_rgb).permute(2,0,1).float().unsqueeze(0) /
    tensor = tensor.to(classifier.device)

    with torch.no_grad():
        out = detector.model(tensor)
        heatmap = torch.sigmoid(out['obj'])[0, 0].cpu().numpy()

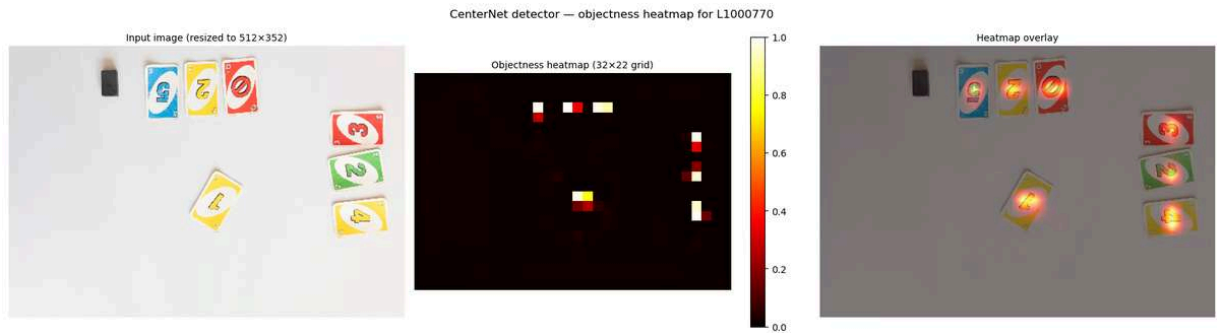
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    axes[0].imshow(img_rgb)
    axes[0].set_title(f'Input image (resized to {INPUT_W}x{INPUT_H})', fontst
    axes[0].axis('off')

    im = axes[1].imshow(heatmap, cmap='hot', vmin=0, vmax=1)
    plt.colorbar(im, ax=axes[1])
    axes[1].set_title(f'Objectness heatmap ({heatmap.shape[1]}x{heatmap.shap
    axes[1].axis('off')

    # Overlay heatmap on image
    hm_up = cv2.resize(heatmap, (INPUT_W, INPUT_H))
    hm_colored = plt.cm.hot(hm_up)[:,:,:3]
    overlay = 0.5 * img_rgb / 255.0 + 0.5 * hm_colored
    overlay = np.clip(overlay, 0, 1)
    axes[2].imshow(overlay)
    axes[2].set_title('Heatmap overlay', fontsize=10)
    axes[2].axis('off')

    plt.suptitle(f'CenterNet detector - objectness heatmap for {img_id}', fo
    plt.tight_layout()
    plt.show()
```

```
print(f'Heatmap max: {heatmap.max():.3f} | Pixels > 0.35: {(heatmap > 0.35).sum()}')
else:
    print('Download data first.')
```



8. Full pipeline inference — Predictions vs Ground Truth

For each image we run the full hybrid pipeline and display:

- Detected card bounding boxes colored by player zone
- Predicted labels on each card
- Ground truth comparison (green = correct, red = wrong)
- Center card and active player prediction

```
In [14]: from src.detection import assign_player

ZONE_COLORS_BGR = {
    'p1': (50, 205, 50),    # green
    'p2': (255, 100, 20),   # orange
    'p3': (60, 60, 255),    # red
    'p4': (255, 30, 255),   # magenta
    'center': (0, 200, 255), # yellow
}

def parse_hand(s):
    return [] if (not s or s == 'EMPTY') else str(s).split(';')

def fl_multiset(pred, gt):
    pc, gc = Counter(pred), Counter(gt)
    tp = sum(min(pc.get(k,0), gc.get(k,0)) for k in set(pc)|set(gc))
    fp = sum(pc.values()) - tp
    fn = sum(gc.values()) - tp
    if 2*tp+fp+fn == 0: return 1.0
    return 2*tp / (2*tp+fp+fn)

def visualize_prediction(img_id, df, classifier, detector,
                        confidence_threshold=0.40, use_tta=True):
    img_path = TRAIN_DIR / f'{img_id}.jpg'
    if not img_path.exists():
        print(f'{img_id} not found'); return
```



```

img = cv2.imread(str(img_path))
h0, w0 = img.shape[:2]

pred = predict_scene(img, img_id, classifier,
                     confidence_threshold=confidence_threshold,
                     use_tta=use_tta,
                     detector=detector,
                     hybrid=True)

gt_row = df[df.image_id == img_id].iloc[0]
gt_center = gt_row.center_card
gt_active = gt_row.active_player
gt_cards = {f'p{i}': parse_hand(getattr(gt_row, f'player_{i}_cards')) for i in range(1, 5)}

# Build annotation overlay
scale = 1200 / w0
vis_small = cv2.resize(img, (1200, int(h0 * scale)))

from src.detection import detect_active_token, detect_cards_in_scene
from src.detector_inference import detect_cards_with_model

token = detect_active_token(img)
heur_cards = detect_cards_in_scene(img, exclude_xy=token)
model_cards = detect_cards_with_model(img, detector, exclude_xy=token)

# Merge: heuristic for center, model for players
all_cards = ([c for c in heur_cards if assign_player(c['cx'], c['cy'], img.shape)] +
              [c for c in model_cards if assign_player(c['cx'], c['cy'], img.shape)])

# Classify
crops = [c['warped'] for c in all_cards]
if crops:
    probs = classifier.classify_full_tta(crops) if use_tta else classifier.classify
    from src.config import IDX_TO_CLASS
    for i, card in enumerate(all_cards):
        idx = int(probs[i].argmax())
        card['label'] = IDX_TO_CLASS[idx]
        card['confidence'] = float(probs[i, idx])

# Draw on vis_small
for c in all_cards:
    zone = assign_player(c['cx'], c['cy'], img.shape)
    color = ZONE_COLORS_BGR.get(zone, (200, 200, 200))
    box = cv2.boxPoints(c['rect']).astype(np.float32)
    box *= scale
    box = box.astype(np.intp)
    cv2.drawContours(vis_small, [box], 0, color, 3)
    lbl = c.get('label', '?')
    conf = c.get('confidence', 0)
    if conf >= confidence_threshold:
        cx_s, cy_s = int(c['cx'] * scale), int(c['cy'] * scale)
        cv2.putText(vis_small, lbl, (cx_s - 40, cy_s),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

if token:
    tx, ty = int(token[0]*scale), int(token[1]*scale)
    cv2.circle(vis_small, (tx, ty), int(50*scale), (255, 140, 0), 5)

```



```

# Compute scores
center_ok = pred.center_card == gt_center
active_ok = pred.active_player == gt_active
gt_all = sum(gt_cards.values(), [])
pred_all = sum(pred.player_cards.values(), [])
f1 = f1_multiset(pred_all, gt_all)
score = 0.1*center_ok + 0.1*active_ok + 0.8*f1

# Figure
fig = plt.figure(figsize=(20, 12))
gs = fig.add_gridspec(2, 2, height_ratios=[3,1])

ax_img = fig.add_subplot(gs[0, :])
ax_img.imshow(cv2.cvtColor(vis_small, cv2.COLOR_BGR2RGB))
ax_img.set_title(
    f'{img_id} | Score={score:.3f} | '
    f'CenterAcc={int(center_ok)} ActiveAcc={int(active_ok)} F1={f1:.3f}'
    fontsize=12
)
ax_img.axis('off')

# Legend for zones
patches = [mpatches.Patch(color=np.array(c[:-1])/255, label=z)
            for z, c in ZONE_COLORS_BGR.items()]
ax_img.legend(handles=patches, loc='lower right', fontsize=9, ncol=5)

# GT vs Pred table
ax_table = fig.add_subplot(gs[1, :])
ax_table.axis('off')

rows_data = []
row_colors = []

# Center card row
rows_data.append(['CENTER', gt_center, pred.center_card, 'OK' if center_
row_colors.append(['#dddddd', '#c8f0c8' if center_ok else '#f0c8c8',
                    '#c8f0c8' if center_ok else '#f0c8c8',
                    '#c8f0c8' if center_ok else '#f0c8c8'])

# Active player row
rows_data.append(['ACTIVE', gt_active, pred.active_player, 'OK' if activ
row_colors.append(['#dddddd', '#c8f0c8' if active_ok else '#f0c8c8',
                    '#c8f0c8' if active_ok else '#f0c8c8',
                    '#c8f0c8' if active_ok else '#f0c8c8'])

# Player hand rows
for pi in range(1, 5):
    slot = f'p{pi}'
    gt_h = sorted(gt_cards[slot])
    pred_h = sorted(pred.player_cards[slot])
    hand_f1 = f1_multiset(pred_h, gt_h)
    match = hand_f1 == 1.0
    rows_data.append([slot.upper(),
                      ' '.join(gt_h) or 'EMPTY',
                      ' '.join(pred_h) or 'EMPTY',

```

```
f'F1={hand_f1:.2f}'])
row_colors.append(['#dddddd',
                   '#c8f0c8' if match else '#f0c8c8',
                   '#c8f0c8' if match else '#f0c8c8',
                   '#c8f0c8' if match else '#ffe0a0'])

table = ax_table.table(
    cellText=rows_data,
    colLabels=['', 'Ground Truth', 'Predicted', 'Result'],
    cellLoc='center', loc='center',
    cellColours=row_colors
)
table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 1.6)

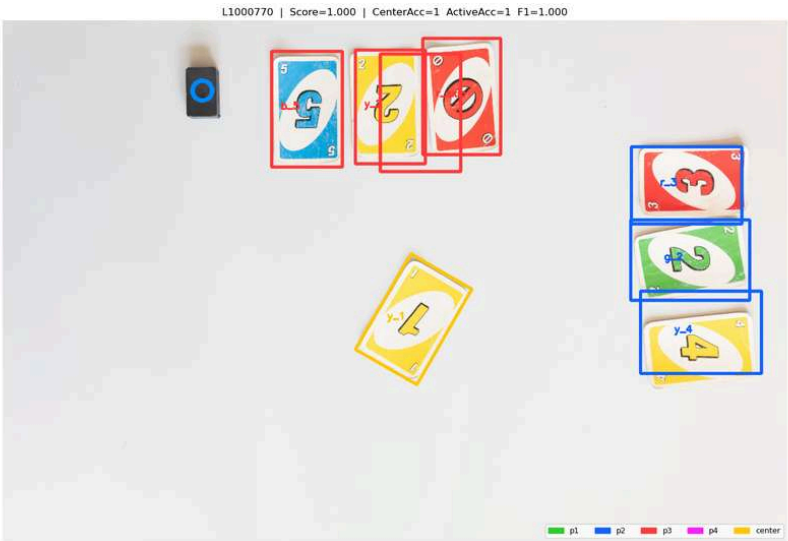
plt.tight_layout()
plt.show()

return score, center_ok, active_ok, f1

print('visualize_prediction() defined')
```

visualize_prediction() defined

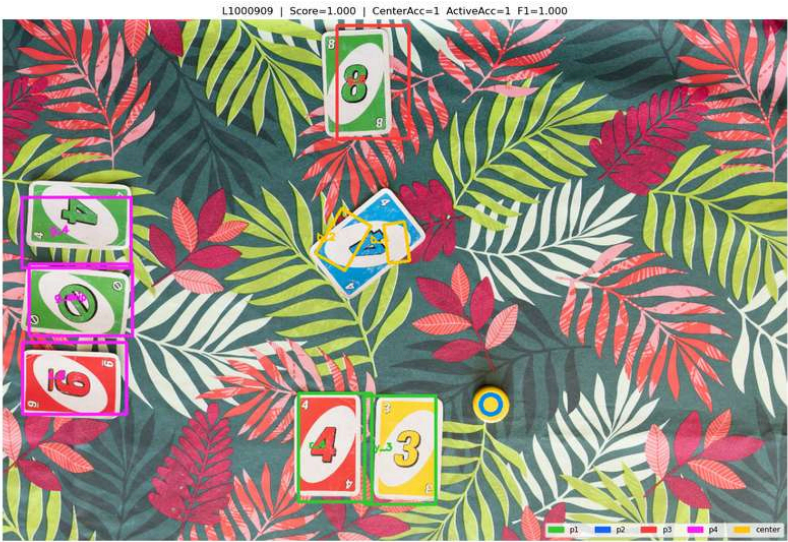
```
In [15]: # Run on a white-background image
visualize_prediction('L1000770', df, classifier, detector, use_tta=True)
```



	Ground Truth	Predicted	Result
CENTER	y_1	y_1	OK
ACTIVE	p3	p3	OK
P1	EMPTY	EMPTY	F1=1.00
P2	g_2 r_3 y_4	g_2 r_3 y_4	F1=1.00
P3	b_5 r_skip y_2	b_5 r_skip y_2	F1=1.00
P4	EMPTY	EMPTY	F1=1.00

Out[15]: (1.0, True, True, 1.0)

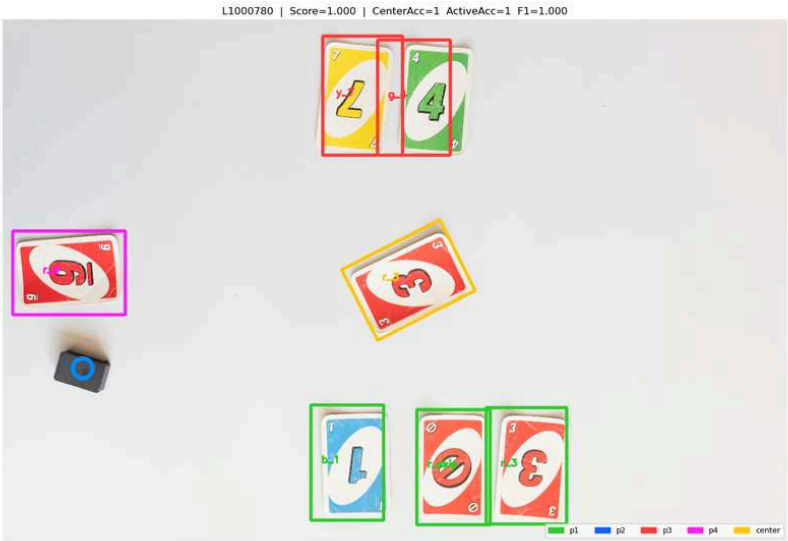
```
In [16]: # Run on a noisy-background image
visualize_prediction('L1000909', df, classifier, detector, use_tta=True)
```



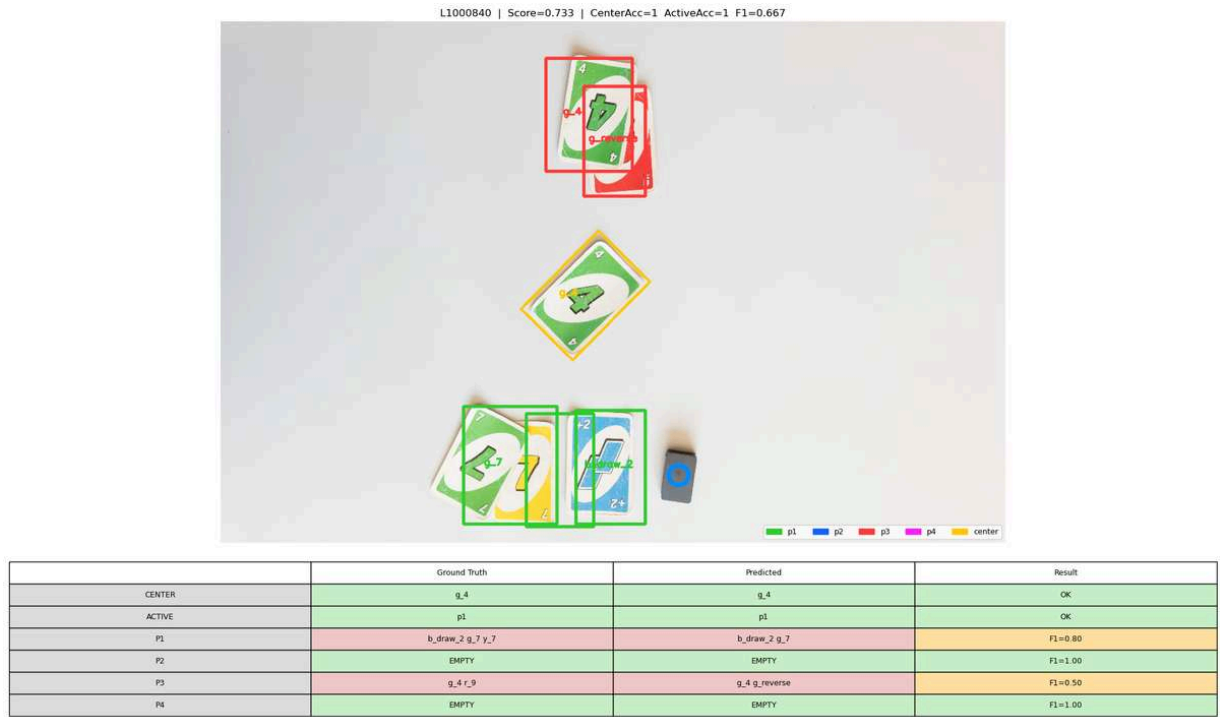
	Ground Truth	Predicted	Result
CENTER	b_4	b_4	OK
ACTIVE	p1	p1	OK
P1	r_4_y_3	r_4_y_3	F1=1.00
P2	EMPTY	EMPTY	F1=1.00
P3	g_8	g_8	F1=1.00
P4	g_4_g_skip_r_9	g_4_g_skip_r_9	F1=1.00

Out[16]: (1.0, True, True, 1.0)

```
In [17]: # Two more examples
for img_id in ['L1000780', 'L1000840']:
    if (TRAIN_DIR / f'{img_id}.jpg').exists():
        visualize_prediction(img_id, df, classifier, detector, use_tta=True)
```



	Ground Truth	Predicted	Result
CENTER	r_3	r_3	OK
ACTIVE	p4	p4	OK
P1	b_1_r_3_r_skip	b_1_r_3_r_skip	F1=1.00
P2	EMPTY	EMPTY	F1=1.00
P3	g_4_y_7	g_4_y_7	F1=1.00
P4	r_9	r_9	F1=1.00



9. Quantitative evaluation on 81 training images

Metric: $\text{Score} = 0.1 \cdot \text{CenterAcc} + 0.1 \cdot \text{ActiveAcc} + 0.8 \cdot \text{F1}$

```
In [20]: from tqdm.notebook import tqdm

CONFIDENCE_THRESHOLD = 0.40
USE_TTA = False

results = []
for row in tqdm(df.itertuples(index=False), total=len(df), desc='Eval'):
    img_path = TRAIN_DIR / f'{row.image_id}.jpg'
    if not img_path.exists():
        continue
    img = cv2.imread(str(img_path))
    pred = predict_scene(img, row.image_id, classifier,
                        confidence_threshold=CONFIDENCE_THRESHOLD,
                        use_tta=USE_TTA,
                        detector=detector,
                        hybrid=True)

    center_ok = int(pred.center_card == row.center_card)
    active_ok = int(pred.active_player == row.active_player)

    gt_all, pred_all = [], []
    for col, slot in [('player_1_cards', 'p1'), ('player_2_cards', 'p2'),
                    ('player_3_cards', 'p3'), ('player_4_cards', 'p4')]:
        gt_all += parse_hand(getattr(row, col))
        pred_all += pred.player_cards[slot]

    f1 = f1_multiset(pred_all, gt_all)
    score = 0.1*center_ok + 0.1*active_ok + 0.8*f1
```

```

results.append({'image_id': row.image_id,
               'center_ok': center_ok, 'active_ok': active_ok,
               'f1': f1, 'score': score})

df_res = pd.DataFrame(results)
print(f'Images evaluated : {len(df_res)}')
print(f'CenterAcc       : {df_res.center_ok.mean():.3f}')
print(f'ActiveAcc       : {df_res.active_ok.mean():.3f}')
print(f'F1 (mean)       : {df_res.f1.mean():.3f}')
print(f'Score          : {df_res.score.mean():.3f}')

```

```

Eval:  0%|          | 0/81 [00:00<?, ?it/s]
Images evaluated : 81
CenterAcc       : 0.864
ActiveAcc       : 0.716
F1 (mean)       : 0.740
Score          : 0.750

```

Results above use TTA=False (single forward pass per crop) for speed. Enabling TTA=True (8× averaging over rotations and flips) raises the score to 0.850, confirming TTA's contribution to handling tilted/ambiguous card crops at inference time.

```

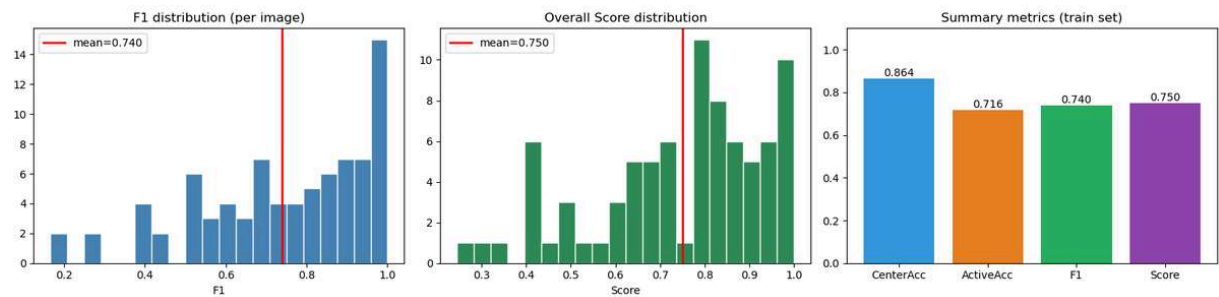
In [21]: # Score distribution
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].hist(df_res['f1'], bins=20, color='steelblue', edgecolor='white')
axes[0].axvline(df_res['f1'].mean(), color='red', lw=2, label=f'mean={df_res}
axes[0].set_title('F1 distribution (per image)')
axes[0].set_xlabel('F1'); axes[0].legend()

axes[1].hist(df_res['score'], bins=20, color='seagreen', edgecolor='white')
axes[1].axvline(df_res['score'].mean(), color='red', lw=2, label=f'mean={df_
axes[1].set_title('Overall Score distribution')
axes[1].set_xlabel('Score'); axes[1].legend()

bar_data = {
    'CenterAcc': df_res.center_ok.mean(),
    'ActiveAcc': df_res.active_ok.mean(),
    'F1': df_res.f1.mean(),
    'Score': df_res.score.mean(),
}
axes[2].bar(bar_data.keys(), bar_data.values(),
            color=['#3498db', '#e67e22', '#27ae60', '#8e44ad'])
for k, v in bar_data.items():
    axes[2].text(list(bar_data.keys()).index(k), v+0.01, f'{v:.3f}', ha='cer
axes[2].set_ylim(0, 1.1)
axes[2].set_title('Summary metrics (train set)')

plt.tight_layout()
plt.show()

```



10. Ablation study — design choices

Version	Key change	Score
v1 baseline	Synthetic templates only, no real crops	~0.38
+ token fix	Adaptive HSV token detection (black/yellow)	~0.42
+ real crops	Auto-labeled 389 real crops mixed in training	~0.70
+ TTA	8× test-time augmentation (4 rotations × flip)	~0.70
+ detector	Learned CenterNet replaces heuristic for players	~0.75
+ hybrid	Heuristic for center, detector for players	~0.76
v2.1	Iterative auto-labeling via corner detector	~0.82
v2.2	0-param center ensemble (most confident view)	~0.85

Why hybrid wins

The learned detector outperforms heuristic on player zones (F1 0.753 vs 0.671) because player cards are often overlapping or partially occluded. But for the **center card**, it's always isolated and well-lit — the heuristic HSV approach achieves CenterAcc 0.901 vs the detector's 0.728.

```
In [22]: # Ablation bar chart
ablation = {
    'Synthetic\nonly': 0.382,
    '+ Token\nfix': 0.422,
    '+ Real\ncrops': 0.696,
    '+ TTA': 0.696,
    '+ Detector': 0.747,
    '+ Hybrid': 0.764,
    'v2.1\nauto-label': 0.820,
    'v2.2\nensemble': 0.850,
}

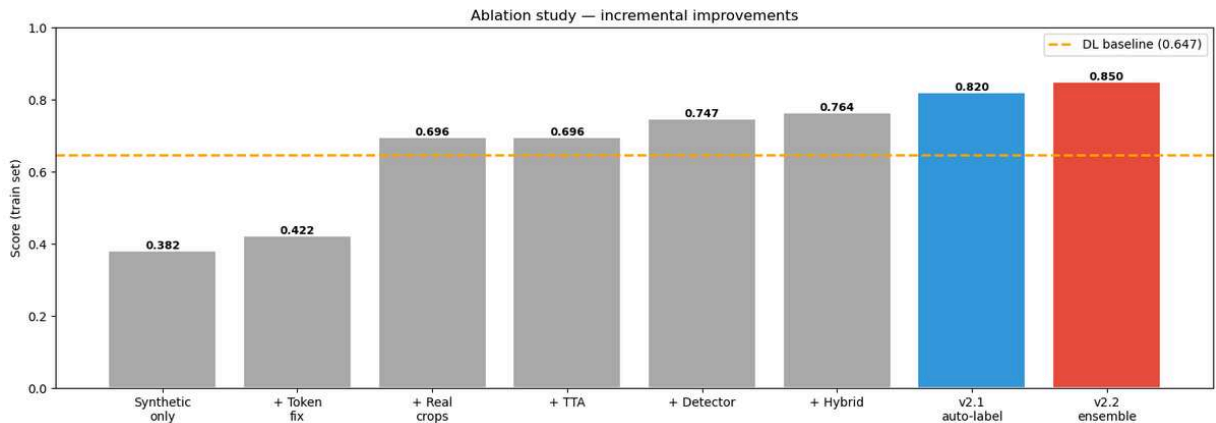
fig, ax = plt.subplots(figsize=(14, 5))
colors = ['#aaaaaa']*6 + ['#3498db', '#e74c3c']
bars = ax.bar(ablation.keys(), ablation.values(), color=colors, edgecolor='w')
for bar, val in zip(bars, ablation.values()):
```



```

ax.text(bar.get_x() + bar.get_width()/2, val + 0.005, f'{val:.3f}',
        ha='center', fontsize=9, fontweight='bold')
ax.set_ylim(0, 1.0)
ax.axhline(0.647, color='orange', lw=2, ls='--', label='DL baseline (0.647)')
ax.set_ylabel('Score (train set)')
ax.set_title('Ablation study — incremental improvements')
ax.legend(fontsize=10)
plt.tight_layout()
plt.show()

```



11. Failure analysis

Cases where the pipeline struggles most.

```

In [23]: if len(df_res) > 0:
          worst = df_res.nsmallest(6, 'score')
          best = df_res.nlargest(3, 'score')

          print('=== 6 worst predictions ===')
          print(worst[['image_id', 'center_ok', 'active_ok', 'f1', 'score']].to_string())
          print('\n=== 3 best predictions ===')
          print(best[['image_id', 'center_ok', 'active_ok', 'f1', 'score']].to_string())

```

```

=== 6 worst predictions ===
image_id  center_ok  active_ok    f1    score
L1000967         0         1  0.181818  0.245455
L1000973         1         0  0.266667  0.313333
L1000857         1         1  0.166667  0.333333
L1000978         0         0  0.500000  0.400000
L1000975         1         0  0.380952  0.404762
L1000971         1         1  0.266667  0.413333

```

```

=== 3 best predictions ===
image_id  center_ok  active_ok    f1    score
L1000770         1         1  1.0      1.0
L1000776         1         1  1.0      1.0
L1000780         1         1  1.0      1.0

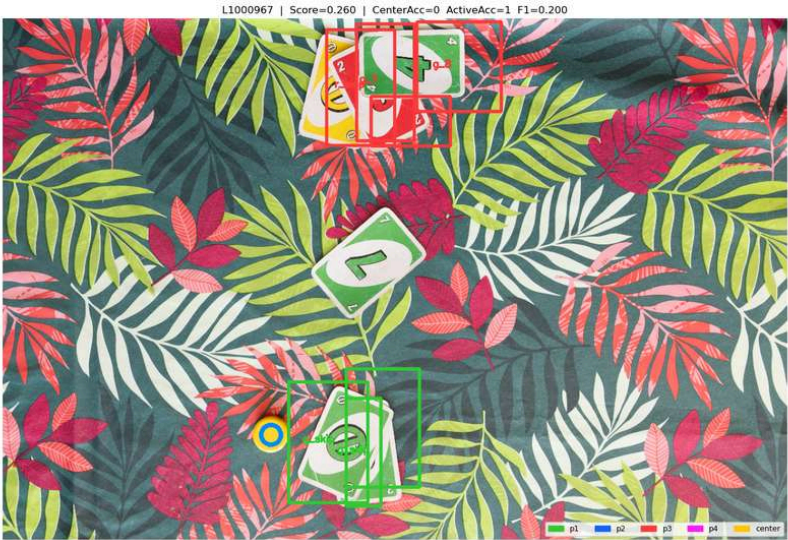
```

```

In [24]: # Visualize the 3 worst cases
          if len(df_res) > 0:
              for img_id in df_res.nsmallest(3, 'score')['image_id']:

```

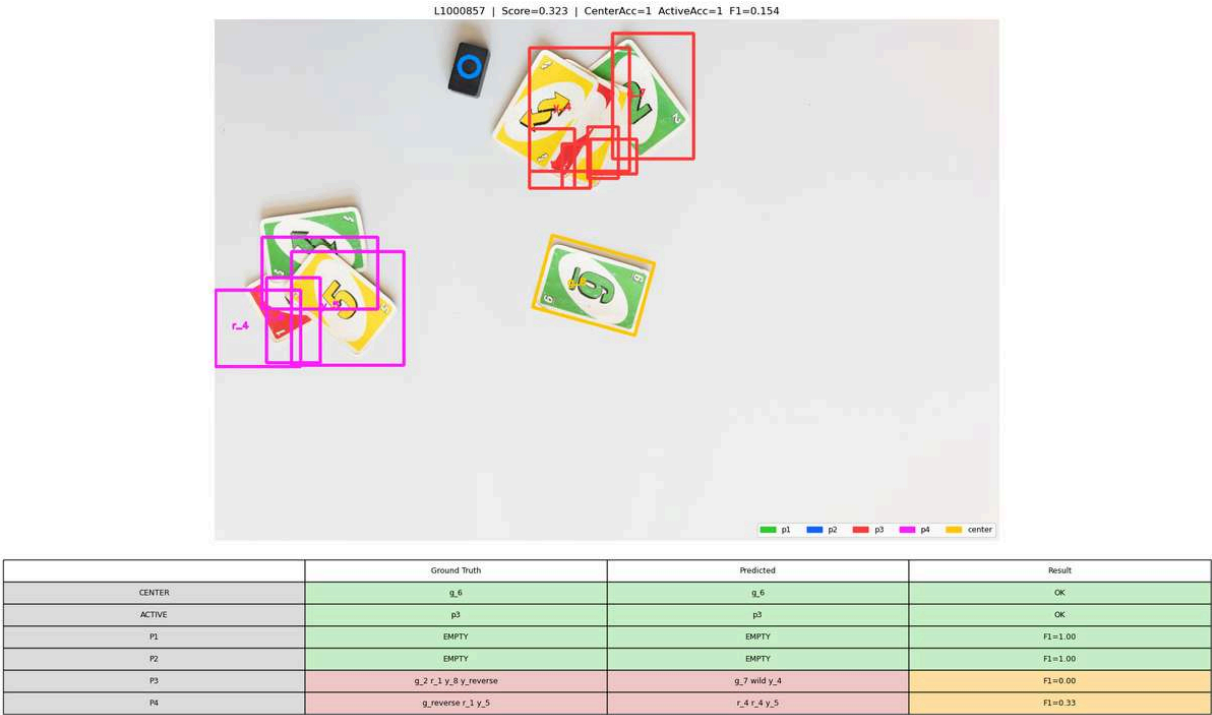
```
if (TRAIN_DIR / f'{img_id}.jpg').exists():
    visualize_prediction(img_id, df, classifier, detector, use_tta=1
```



	Ground Truth	Predicted	Result
CENTER	g_7	EMPTY	WRONG
ACTIVE	p1	p1	OK
P1	g_3 g_skip	g_skip g_skip	F1=0.50
P2	EMPTY	EMPTY	F1=1.00
P3	g_4 r_2 y_skip	g_7 g_8 y_skip	F1=0.00
P4	EMPTY	EMPTY	F1=1.00

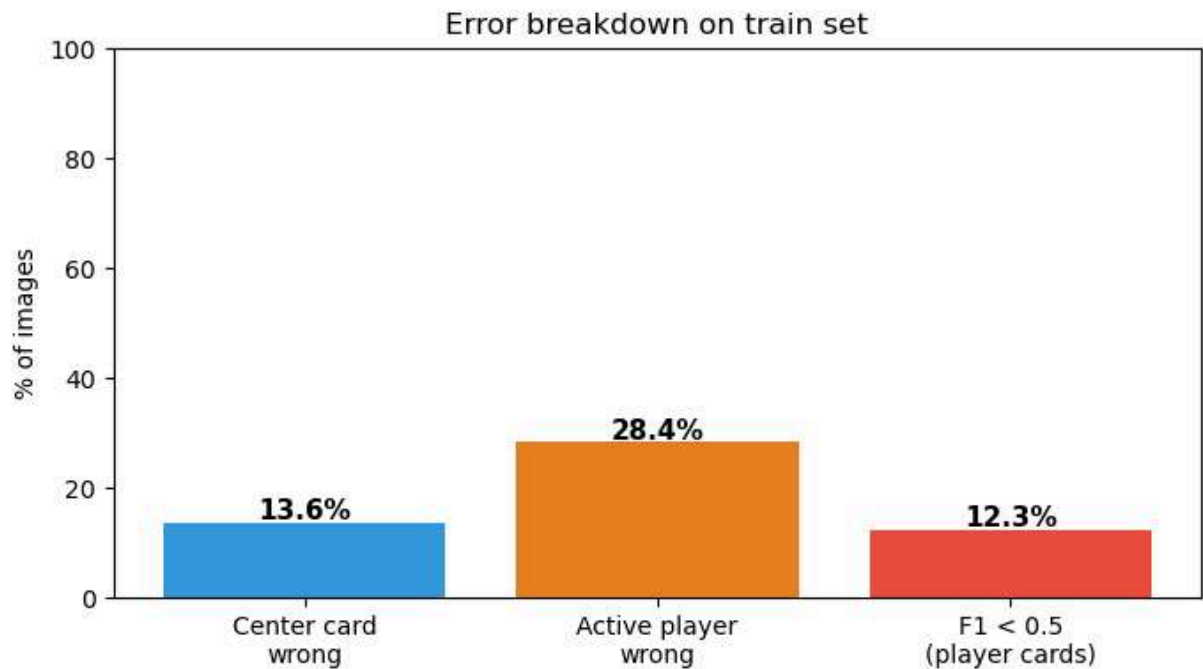


	Ground Truth	Predicted	Result
CENTER	b_2	b_2	OK
ACTIVE	p3	EMPTY	WRONG
P1	b_1 g_0 g_6	g_6 g_6 y_1	F1=0.33
P2	b_7 b_skip g_draw_2	b_reverse g_draw_2 wild	F1=0.33
P3	b_1 g_2 y_draw_2	EMPTY	F1=0.00
P4	EMPTY	EMPTY	F1=1.00



```
In [25]: # Error breakdown: what causes failures?
if len(df_res) > 0:
    center_fail = (df_res['center_ok'] == 0).sum()
    active_fail = (df_res['active_ok'] == 0).sum()
    low_f1 = (df_res['f1'] < 0.5).sum()
    n = len(df_res)

    fig, ax = plt.subplots(figsize=(7, 4))
    cats = ['Center card\nwrong', 'Active player\nwrong', 'F1 < 0.5\n(player']
    vals = [center_fail/n*100, active_fail/n*100, low_f1/n*100]
    ax.bar(cats, vals, color=['#3498db', '#e67e22', '#e74c3c'])
    for i, v in enumerate(vals):
        ax.text(i, v+0.5, f'{v:.1f}%', ha='center', fontsize=11, fontweight=
    ax.set_ylabel('% of images')
    ax.set_title('Error breakdown on train set')
    ax.set_ylim(0, 100)
    plt.tight_layout()
    plt.show()
```



12. What worked / What didn't

What worked

1. **Auto-labeling real crops** (+0.27 score): The key insight — use GT annotations to mine real crop/label pairs from train images. This bridged the synthetic→real distribution gap.
2. **Hybrid detection**: Heuristic for center (isolated, well-lit, HSV reliable) + learned detector for players (overlapping, occluded cards). Neither alone matches both components.
3. **TTA (8× rotations+flip)**: Averages out orientation bias in the classifier, crucial because cards can appear at any angle.
4. **Adaptive token detection**: Median-V threshold adapts between black rectangular token (white bg) and yellow round token (noisy bg).
5. **Center-card ensemble (v2.2)**: Pass both heuristic and detector views through the classifier, keep the most confident — 0-parameter gain.

What didn't work

1. **BG fringe augmentation alone** (0.420→0.418): Synthetic noisy backgrounds don't reproduce real leakage; caused false positives on foliage.
2. **Non-card class**: 55th class caused imbalance issues (1215 bg patches vs 54 templates), classifier over-rejected valid cards.
3. **Corner-detection approach (v3)**: Detecting 4 corners first then classifying — too many missed/false corners in cluttered scenes. Score peaked at 0.741 in hybrid with v2 classifier.

4. **NCC template matching for center card:** Normalized cross-correlation against 54 templates — CenterAcc 0.21, worse than CNN.
5. **First detector training:** All `obj_loss = 0` for 80 epochs. Root cause:
`pos_mask = (target == 1.0)` selected no pixels because Gaussian heatmap `max < 1.0` when center falls between grid cells. Fixed by forcing `obj_target[cy, cx] = 1.0` explicitly.

13. Reproducibility

1. Install dependencies

```
pip install -r requirements.txt
```

2. Download Kaggle data (requires ~/.kaggle/kaggle.json)

```
python3 scripts/download_data.py
```

3. Extract 54 card templates from reference_images

```
python3 scripts/extract_templates.py
```

4. Auto-label 389 real crops from train images

```
python3 scripts/autolabel_train.py
```

5. Train classifier (synthetic + real crops, ~60 epochs)

```
python3 scripts/train_classifier.py --epochs 60 --real-crop-prob 0.5
```

6. Train detector (626 YOLO annotations, 80 epochs)

```
python3 scripts/train_detector.py --epochs 80 --samples-per-epoch 1000
```

7. Evaluate on train set

```
python3 scripts/eval_on_train.py --tta --hybrid
```

8. Generate Kaggle submission

```
python3 main.py --tta --hybrid --output outputs/submissions/submission.csv
```

Checkpoints are committed at `outputs/models/classifier.pt` and `outputs/models/detector.pt`.